SDC Hackathon 2026

Phase 1: Preliminary Project Proposal — Vortex OrderGuard

Selected Domain: Domain 2 — Orbyte Smart SME Challenge: Less Busywork, Better Business
Selected SDG: SDG 8 — Decent Work and Economic Growth; intended alignment with Targets 8.2 and 8.3
Live prototype: https://sdc-hackathon-submission-template.replit.app
Repository: https://github.com/Aunyiqi/SDC_Hackathon_Submission_Template
Privacy note: leader email/phone and signature are intentionally omitted from this public GitHub copy; complete them in the privately submitted official copy.

Team Vortex · Vortex OrderGuard

Prepared 24 October 2026

1. Team & Project Details

Team Name: Vortex

Project / Solution Name: Vortex OrderGuard

Selected Domain: Domain 2

Selected SDG Track: SDG 8

Team Leader: Aun Yi Qi

University Affiliation: Multimedia University (MMU)

Number of Team Members: 4

Leader Email & Phone: Omitted from public repository copy; complete in the official private submission.

2.1. The Problem Context

Micro and small businesses often reconcile purchase orders, receiving records and supplier invoices manually across messages, spreadsheets and documents. A quantity, unit or price mismatch can be difficult to notice quickly, and the owner may spend time checking evidence before deciding whether to contact a supplier.

OrderGuard focuses on this review bottleneck. The economic relevance is broad: Malaysia’s MSMEs account for a large share of national output and employment, so practical productivity tools for small firms matter. The prototype does not claim that every SME experiences the same error rate; real SME interviews and measured baseline timing remain required validation work.

Research context: DOSM, MSMEs Performance 2025 (released 30 July 2026); World Bank, Digitalizing SMEs to Boost Competitiveness (2022).

2.2. The Proposed AI Solution

Vortex OrderGuard is an AI-assisted supplier discrepancy review workflow. Users provide supplier text and structured purchasing evidence. Optional AI/OCR extracts suggestions from unstructured text or invoice images, while deterministic rules compare purchase order, receiving and invoice values. The system highlights quantity, unit and price exceptions, translates open discrepancies into a visible Money at Risk signal, remembers repeat supplier-item issues, and uses a transparent Trust Gate to classify cases as CLEAR, REVIEW or BLOCKED.

AI never approves, rejects, converts ambiguous units or sends supplier messages. Ambiguous evidence is blocked until a human clarifies it, and consequential actions require owner approval. Workshop-aligned skills used include GenAI/API integration, database-backed workflow design, cloud deployment through Replit, and GPU serverless OCR through RunPod.

3.1. Conceptual Tech Stack

Architecture: Browser UI → Flask application → optional DeepSeek extraction / RunPod OCR → human-confirmed structured fields → deterministic discrepancy engine → Trust Gate + Supplier Pattern Memory → owner approval/rejection → simulated outbox + audit history → SQLite demo database.

Official sponsor-provided platform/resource used: RunPod sponsor credit was used for GPU serverless OCR proof-of-use. Replit is used for the required live deployment. No live Orbyte API integration is claimed.

Component	Technology
Frontend	HTML, CSS, JavaScript
Backend	Python 3, Flask
Data	SQLite demo database
AI text extraction	DeepSeek API (optional)
OCR	RunPod Serverless + EasyOCR
Hosting	Replit
CI / repository	GitHub + GitHub Actions

3.2. Design-Thinking & Deployment Strategy

The team defined a narrow operational task: reviewing supplier discrepancies without giving AI authority over business decisions. The design prioritizes explainability, visible evidence and human control. Four synthetic judge scenarios exercise discrepancy, repeat issue, ambiguous unit and clean match flows. The public demo defaults to read-only mode; owner controls require a private time-limited access code.

Deployment was tested on Replit and published as a public HTTPS application. Automated regression tests cover matching, discrepancy rules, Trust Gate blocking, owner-only actions and security controls. Phase 2 validation should add SME interviews, measured task timing and usability testing before making productivity claims.

4. Impact Baseline & Evidence

The project uses published SME economic context and a prototype test baseline rather than claiming measured field impact. DOSM reported MSMEs contributed 39.7% of Malaysia’s GDP and 48.7% of employment in 2025. This supports the importance of SME productivity but is not a measured OrderGuard customer baseline.

For Phase 2, the team will establish a controlled manual-vs-prototype baseline using the same known supplier cases and record median review time, discrepancy detection rate and false alerts.

4. Impact Metrics — Part 1

Metric Category	Response
Target SDG Sub-Target(s)	SDG 8.2: higher productivity through diversification, technological upgrading and innovation; SDG 8.3: support productive activities,
Current Problem Baseline	Pilot baseline to be measured. Initial controlled baseline assumption: current target workflow requires manual review of 100% of sup

4. Impact Metrics — Part 2

Metric Category	Response
Core Measurable KPI	Median review time per supplier check (minutes), known discrepancy detection rate (%), false-alert count on clean cases, first-time-use
Phase 2 Impact Target	Target at least 30% reduction in median review time on the same controlled test set while maintaining 100% detection of seeded discre

5. Preliminary Checklist

• [ ] Mandatory GenAI, Database and Cloud Deployment workshop completion — team to confirm in private submission.

• [ ] Verified real-world problem — public research supports the context; direct SME validation is still pending and should be completed before claiming verification.

• [x] Solution targets a functional deployed application, not a theoretical business case.

• [x] Responses in this public copy are written within the stated word limits.

6.1. Pre-Existing / Prior-Owned Work

Answer: No team-owned pre-existing product code is claimed as part of Vortex OrderGuard. The repository originated from the official SDC starter/submission template and uses declared open-source dependencies. The OrderGuard workflow, rules, UI and documentation were developed for the hackathon.

6.2. Third-Party Code, Datasets, or Models

Name	Type	License / Terms
Flask	Code framework	BSD-3-Clause
Requests	Code library	Apache-2.0
Gunicorn	Code server	MIT
RunPod Python SDK	Code / cloud SDK	MIT
EasyOCR 1.7.2	Pre-trained OCR library/model	Apache-2.0
OpenCV / Pillow / NumPy	OCR dependencies	Upstream open-source licenses
DeepSeek API (optional)	External AI API	Provider terms
Synthetic demo records	Dataset	Team-created synthetic data; no personal data

6.3. Employment-Related IP Conflicts

No employer-owned code, confidential information or employer systems are intentionally included in this project. The public repository contains hackathon-specific work and synthetic data. Any team member with a separate internship or employment agreement must independently confirm that agreement before final private submission.

7. Team Leader Confirmation

The team leader confirms that the information in the final privately submitted copy is accurate to the best of the team’s knowledge and that required personal contact/signature fields will be completed there.

Signature: Omitted from public GitHub copy.

Date: 24 October 2026